

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 14 实验名称 网络传输机制实验-可靠传输

一、实验内容

1. 执行 `create_randfile.sh`, 生成待传输数据文件 `client-input.dat`
2. 运行给定网络拓扑(`tcp_topo_loss.py`)
3. 在节点 `h1` 上执行 TCP 程序
 - (a) 执行脚本(`disable_offloading.sh`, `disable_tcp_rst.sh`), 禁止协议栈的相应功能
 - (b) 在 `h1` 上运行 TCP 协议栈的服务器模式(`./tcp_stack server 10001`)
4. 在节点 `h2` 上执行 TCP 程序
 - (a) 执行脚本(`disable_offloading.sh`, `disable_tcp_rst.sh`), 禁止协议栈的相应功能
 - (b) 在 `h2` 上运行 TCP 协议栈的客户端模式(`./tcp_stack client 10.0.0.1 10001`)
 - i. 客户端发送文件 `client-input.dat` 给服务器, 服务器将收到的数据存储在文件 `server-output.dat`
5. 使用 `md5sum` 比较两个文件是否完全相同
6. 使用 `tcp_stack.py` 替换两端任意一方, 对端都能正确处理数据收发

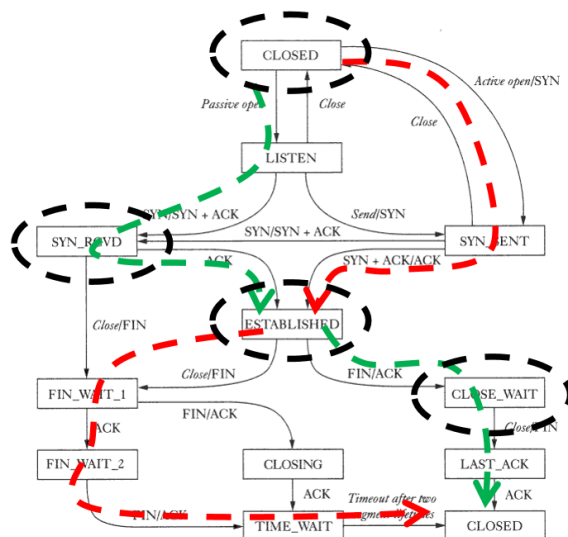


图 1: TCP 状态机图

二、 实验步骤

2.1 数据结构设计

为了支持发送队列和乱序接收队列的管理，定义了新的数据结构 `struct data_packet`，用于存储数据包的相关信息。同时在 `struct tcp_sock` 中增加了发送队列 `send_buf` 和乱序接收队列 `rcv_ofo_buf` 及其对应的互斥锁。

Listing 1: 数据结构定义

```
struct data_packet {
    struct list_head list;
    u8 flags;
    u8 times;    // 重传次数
    u32 seq;
    u32 len;
    u32 seq_end;
    char *packet; // 数据包内容
};

struct tcp_sock {
    // ... existing code ...
    // used to pend unacked packets
    struct list_head send_buf;
    // used to pend out-of-order packets
    struct list_head rcv_ofo_buf;

    pthread_mutex_t send_buf_lock;
    pthread_mutex_t rcv_buf_lock;
    // ... existing code ...
};
```

此外，为了支持微秒级的重传定时器，修改了 `struct tcp_timer` 中的 `timeout` 字段为 `long long` 类型。

2.2 发送队列维护

2.2.1 加入发送队列

实现了 `new_data_block` 函数用于构建 `data_packet` 结构体。在 `tcp_sock_connect` (SYN)、`tcp_sock_close` (FIN) 和 `tcp_sock_write` (Data) 等发送数据的函数中，调用该函数将数据包加入 `send_buf` 队列，并启动重传定时器。

```
// 示例：在 tcp_sock_write 中加入发送队列
struct data_packet *dp = new_data_block(TCP_PSH|TCP_ACK, tsk->snd_nxt, data_len, buf + len_send);
pthread_mutex_lock(&tsk->send_buf_lock);
list_add_tail(&dp->list, &tsk->send_buf);
pthread_mutex_unlock(&tsk->send_buf_lock);

// 发送数据包
tcp_send_packet(tsk, packet, packet_len);
```

```
// 启动重传定时器
if(!tsk->retrans_timer.enable)
    tcp_set_retrans_timer(tsk);
```

2.2.2 清除已确认数据

实现了 `tcp_free_send_buf` 函数。当收到 ACK 时,在 `tcp_process` 中调用该函数,遍历 `send_buf`,删除 `seq_end` 小于等于 `ack` 的数据包,并更新定时器状态。

```
void tcp_free_send_buf(struct tcp_sock *tsk, struct tcp_cb *cb)
{
    struct data_packet *tmp, *q;
    pthread_mutex_lock(&tsk->send_buf_lock);
    list_for_each_entry_safe(tmp, q, &tsk->send_buf, list)
    {
        if(less_or_equal_32b(tmp->seq_end, cb->ack))
        {
            list_delete_entry(&tmp->list);
            if(tmp->packet) free(tmp->packet);
            free(tmp);
            // 重置定时器逻辑...
        }
    }
    pthread_mutex_unlock(&tsk->send_buf_lock);
}
```

2.3 乱序接收处理

2.3.1 缓存乱序包

实现了 `tcp_rcv_ofo_pkt` 函数。当接收到的数据包序列号不等于 `rcv_nxt` 时,调用该函数将数据包按序列号顺序插入 `rcv_ofo_buf` 队列,并回复 ACK(携带期望的 `rcv_nxt`)。

```
void tcp_rcv_ofo_pkt(struct tcp_sock *tsk, struct tcp_cb *cb)
{
    struct data_packet *dp = new_data_block(cb->flags, cb->seq, cb->pl_len, cb->payload);
    // ... 遍历队列找到合适位置插入 ...
    list_add_tail(&dp->list, pos);
}
```

2.3.2 处理接收队列

在 `tcp_process` 中,当收到符合 `rcv_nxt` 的数据包后,将其写入接收缓冲区,随后检查 `rcv_ofo_buf`。如果队列头部的数据包序列号等于更新后的 `rcv_nxt`,则将其取出并写入接收缓冲区,重复此过程直到队列为空或序列号不连续。

```
// 检查乱序队列
list_for_each_entry_safe(tmp, q, &tsk->rcv_ofo_buf, list) {
    if(tmp->seq == tsk->rcv_nxt) {
```

```

    // ... 写入接收缓冲区 ...
    tsk->rcv_nxt = tmp->seq_end;
    // ... 处理 FIN 标志 ...
    list_delete_entry(&tmp->list);
    free(tmp);
} else {
    break;
}
}

```

2.4 超时重传机制

2.4.1 定时器管理

实现了 `tcp_set_retrans_timer` 和 `tcp_unset_retrans_timer` 函数用于开启和关闭重传定时器。定时器类型设为 1。

2.4.2 超时处理与指数避退

修改了 `tcp_scan_timer_list` 函数。当检测到重传定时器超时：1. 检查重传次数，若超过 3 次，则发送 RST 并关闭连接。2. 若未超过次数，则重传队首数据包，并将超时时间翻倍（指数避退）。3. 调用 `tcp_send_retrans_packet` 发送数据包（不增加 `snd_nxt`）。

```

// 指数避退计算
long long duration = TCP_RETRANS_INTERVAL_INITIAL * (1 << dp->times);
if (cur_time - entry->timeout > duration) {
    if (dp->times >= 3) {
        // ... 发送 RST 并关闭连接 ...
    } else {
        dp->times++;
        entry->timeout = cur_time;
        tcp_send_retrans_packet(tsk, dp);
    }
}

```

三、实验结果

先调用 `create_randfile.sh` 生成待传输数据文件 `client-input.dat`，然后运行给定的网络拓扑脚本 `tcp_topo_loss.py`。在节点 `h1` 上执行 TCP 协议栈的服务器模式，并在节点 `h2` 上执行客户端模式。客户端发送文件后，服务器将收到的数据存储到文件 `server-output.dat`。

```

Routing table of 1 entries has been loaded.
DEBUG: 0.0.0.0:10001 switch state, from CLOSED to LISTEN.
DEBUG: listen to port 10001.
DEBUG: 10.0.0.1:10001 switch state, from LISTEN to SYN_RECV.
DEBUG: 10.0.0.1:10001 switch state, from SYN_RECV to ESTABLISHED.
DEBUG: accept a connection.
DEBUG: start to receive data.
DEBUG: 10.0.0.1:10001 switch state, from ESTABLISHED to CLOSE_WAIT.
DEBUG: 10.0.0.1:10001 switch state, from CLOSE_WAIT to LAST_ACK.
DEBUG: 10.0.0.1:10001 switch state, from LAST_ACK to CLOSED.

```

图 2: h1 log 输出

```

Routing table of 1 entries has been loaded.
DEBUG: 10.0.0.2:12345 switch state, from CLOSED to SYN_SENT.
DEBUG: 10.0.0.2:12345 switch state, from SYN_SENT to ESTABLISHED.
DEBUG: start to send data.
DEBUG: data length = 4052632.
DEBUG: 10.0.0.2:12345 switch state, from ESTABLISHED to FIN_WAIT-1.
DEBUG: 10.0.0.2:12345 switch state, from FIN_WAIT-1 to FIN_WAIT-2.
DEBUG: 10.0.0.2:12345 switch state, from FIN_WAIT-2 to TIME_WAIT.
DEBUG: 10.0.0.2:12345 switch state, from TIME_WAIT to CLOSED.

```

图 3: h2 log 输出

使用 md5sum 比较两个文件,结果显示两者完全相同,验证了可靠传输的正确性。

```

kouyixin@ubuntu:~/CNLab/14-tcp_stack/code$ md5sum *.dat
2f56ce9b0a9267d7a765f1dd37f76c71  client-input.dat
2f56ce9b0a9267d7a765f1dd37f76c71  server-output.dat

```

图 4: md5sum 比较结果

四、实验总结

本次实验代码量并不大,难点在于如何构建发送队列和乱序接收队列,以及如何处理超时重传机制(包括状态机的修改,缓存包和释放的时机)。实验结果表明,修改后的 TCP 协议栈能够正确处理数据传输,并且在丢包和乱序情况下仍能保证数据的完整性和顺序。

在这次实验中遇到了一个重复释放的问题,这是由于在处理 TCP_LISTEN 状态收到 SYN 包时,使用了 memcpy 将父 socket (tsk) 的内容复制到了新分配的子 socket (child_tsk) 中引起的。alloc_tcp_sock 为 child_tsk 分配的资源被 memcpy 覆盖,导致 child_tsk 和 tsk 共享同一个 rcv_buf,从而引发了双重释放错误。解决方案是避免使用 memcpy,手动初始化子 socket,仅复制必要字段,确保父子 socket 的资源独立。